Análisis de soluciones

1. Hello PyConES

Solo hay que escribir `Hello, PyConES!` con `print`. No hay entrada y la salida debe ser exacta: mayúsculas, coma, espacio y exclamación.

2. PyRoom to Grow

Cada expansión duplica los asientos, así que tras `n` expansiones hay `2^n`. Como `0 ≤ n ≤ 30`, el máximo es `2^30 = 1073741824`. Proponemos estas soluciones:

1. **Ciclo:** empezar con `1` y multiplicar por 2, `n` veces.
2. **Potencia:** `2 ** n`.
3. **Desplazamiento de bits:** `1 << n`.

Caso borde: con `n = 0` la respuesta es `1`.

3. Pythonic Art

Se lee la línea, se separa en palabras y se convierten las medidas a enteros. Según la figura:

- **`triangle a b c`:** perímetro `a + b + c`.
- **`square l`:** perímetro `4 * l`.

4. Python in the Air

Hay que contar las apariciones de `Python` (distingue mayúsculas y minúsculas). Con como máximo `10^5` caracteres y un patrón de 6, la fuerza bruta basta. Proponemos estas soluciones:

1. **Método `count`:** `s.count("Python")`. No cuenta apariciones solapadas, pero aquí da igual: `Python` no tiene ningún prefijo propio que sea también sufijo, así que no puede solaparse consigo mismo.
2. **Ciclo por posiciones:** para cada `i`, comprobar si los 6 caracteres que empiezan en `i` son `Python` y sumar 1.

5. PyCoffee

Hay que encontrar el nombre que más se repite; en caso de empate, el primero en orden alfabético (como texto, por eso `dev10` va antes que `dev2`). Con `n ≤ 10^5` basta una solución `O(n)` o `O(n log n)`. Proponemos estas soluciones:

1. **Diccionario de conteo:** contar cada nombre y elegir el de más apariciones, desempatando por el menor alfabéticamente. Coste `O(n)`.
2. **Ordenar la lista:** los nombres iguales quedan juntos. Se recorre contando la racha actual y se actualiza la respuesta solo si la racha es **estrictamente mayor** que la mejor. Así, en un empate se conserva el primero alfabéticamente. Coste `O(n log n)`.

6. Always in PySync

Los ciclos coinciden en los múltiplos comunes de `a` y `b`, así que el primer minuto en que se alinean es el **mínimo común múltiplo** (mcm). Como `a, b ≤ 1000`, el resultado es como máximo `999000`. Proponemos estas soluciones:

1. **Fuerza bruta:** recorrer `a`, `2*a`, `3*a` ... hasta encontrar un múltiplo divisible por `b`. Como mucho son `1000` iteraciones.
2. **Fórmula con el mcd:** `a * b // math.gcd(a, b)`.

7. The PyRate Radio

Es un cifrado César: cada letra se desplaza `k` posiciones y vuelve al inicio tras la `z`. Mayúsculas y minúsculas se conservan, y el resto de caracteres no cambia. Como `k` puede llegar a `10^6` pero el alfabeto tiene 26 letras, desplazar `k` es lo mismo que desplazar `k % 26`. Con `len(S) ≤ 500` se puede recorre el texto completo. Proponemos estas soluciones:

1. **Alfabeto como cadena:** guardar el alfabeto en minúsculas y en mayúsculas. Si `c` está en uno, se busca su posición `idx` con `find` y se toma la letra en `(idx + k) % 26`.
2. **`ord` y `chr`:** para una minúscula, `chr(ord('a') + (ord(c) - ord('a') + k) % 26)`; para mayúsculas, igual con `'A'`. Antes hay que comprobar que `c` está en `'a'..'z'` o `'A'..'Z'`.

8. PyConcurrency Issues

Es el clásico problema de **selección de actividades**: elegir el máximo de charlas sin solapes. Se resuelve con un algoritmo **greedy**: elegir siempre la charla que **termina antes** entre las compatibles con las ya elegidas, porque es la que deja más tiempo libre para las siguientes. Si una charla termina justo cuando empieza otra, no hay solape: `(start, end)` es compatible si `start >= fin_última`. Con `n ≤ 10^5` basta `O(n log n)`. Proponemos estas soluciones:

1. **Ordenar por hora de fin:** ordenar las charlas por `end` de menor a mayor, de modo que siempre se consideren primero las que terminan antes. Después, recorrerlas en ese orden con una variable `fin` (empieza en `0`). Si `start >= fin`, se elige: se suma 1 y `fin = end`. Coste `O(n log n)`, por la ordenación.
2. **Agrupar por hora de fin:** como `end ≤ 10^4`, se guardan las charlas en una lista indexada por `end` y se recorren los minutos de `0` a `10^4` en orden, lo que equivale a tenerlas ordenadas por fin. Para cada charla se hace la misma comprobación `start >= fin`. Coste `O(n + 10^4)`, sin `sort`.

9. PyConcurrency Issues II

Ahora hay que cubrir **todas** las charlas con el mínimo de personas. La respuesta es el **máximo de charlas que coinciden a la vez**:

- **Hacen falta al menos ese número:** si hay `k` charlas activas a la vez, cada una necesita una persona distinta.
- **Con ese número basta:** al recorrer las charlas por inicio, solo se necesita una persona nueva cuando todas están ocupadas, y eso nunca supera el máximo de simultáneas.

Si una charla termina en `t` y otra empieza en `t`, no coinciden: la persona queda libre. Por tanto, el problema se reduce a calcular, en cada instante, cuántas charlas están activas, y quedarnos con el máximo. Con `n ≤ 10^5` y tiempos hasta `10^4`, proponemos estas soluciones:

1. **Array de diferencias:** crear un array `a` con una posición por minuto, de `0` a `10^4`. Por cada charla, sumar `1` en `a[start]` (empieza una charla) y restar `1` en `a[end]` (termina). Después, recorrer el array acumulando la suma: el valor acumulado en el minuto `t` es el número de charlas activas en ese instante. La respuesta es el mayor valor acumulado. Como el fin se resta en el mismo minuto en que otra puede empezar, el caso `start == end` queda bien tratado. Coste `O(n + 10^4)`, sin ordenar.
2. **Dos punteros:** guardar por separado la lista de inicios y la de finales, y ordenar ambas. Con un puntero en cada lista, se procesa siempre el evento más próximo en el tiempo. Se mantiene un contador de charlas activas y el máximo visto hasta ahora:
   - Si el siguiente inicio es **estrictamente anterior** al siguiente final, empieza una charla: se suma `1` al contador.
   - Si no (el final es anterior o coincide con el inicio), termina una charla: se resta `1`.

   Tras cada suma se actualiza el máximo. Se puede parar cuando se acaban los inicios, porque a partir de ahí el contador solo baja. Coste `O(n log n)`, por la ordenación.

10. PyConES's Broken Search Bar

Hay que contar, por consulta, cuántas palabras empiezan por un prefijo. Con `n, q ≤ 10^5`, comparar cada consulta con todas las palabras costaría unas `10^10` operaciones. La idea común es **preprocesar** las palabras una sola vez. Proponemos estas soluciones:

1. **Diccionario de prefijos:** guardar cuántas veces aparece cada prefijo de cada palabra. Es la más fácil de implementar.
2. **Hash de prefijos:** igual, pero guardando un hash polinómico `h = (h * B + ord(c)) % MOD` en vez de la cadena. Esta solución ahorra memoria. Para evitar colisiones se usan dos hashes, y la consulta debe usar exactamente los mismos parámetros.
3. **Ordenar y búsqueda binaria:** al ordenar, las palabras con el mismo prefijo quedan juntas. Con búsqueda binaria se halla la primera posición `ini` con `palabra[:l] >= prefijo` y la primera `fin` con `palabra[:l] > prefijo`; la respuesta es `fin - ini`.

4. **Trie:** un árbol donde cada arista es una letra y cada nodo guarda cuántas palabras pasan por él. Para consultar se baja por el prefijo y se devuelve el contador del último nodo (o `0` si falta una arista). Es la estructura pensada para este problema, aunque también puede tener hasta `5*10^6` nodos.

11. PyLinkedIn

Cada persona es un nodo y cada presentación una arista. Por la Py-transitividad, la red de una persona es su **componente conexa**, así que hay que calcular el **tamaño de la componente** de cada persona preguntada. Detalles:

- Quien no aparece en ninguna presentación forma una componente de tamaño `1`.
- Una pareja repetida no cambia nada.
- Las consultas se responden tras procesar **todas** las presentaciones.

Con `n, m, q ≤ 2*10^5`, recorrer el grafo en cada consulta es demasiado lento, así que se calculan los tamaños una vez. En ambas soluciones se asigna primero un número a cada nombre con un diccionario. Proponemos estas soluciones:

1. **BFS o DFS:** construir la lista de adyacencia, recorrer cada componente una vez y guardar su tamaño para todos sus nodos. Coste `O(n + m)`. Si se usa DFS, hay que hacerlo iterativo (con pila) o subir el límite con `sys.setrecursionlimit`, porque una cadena de `2*10^5` personas supera el límite de recursión.
2. **Union-Find (disjoint set):** cada persona empieza en su propio conjunto y, por cada presentación, se unen los conjuntos si son distintos. Con **unión por tamaño** y **compresión de caminos**, cada operación es casi constante. Para responder, se busca el representante y se imprime el tamaño de su conjunto. No hace falta construir el grafo.

12. The Galactic Pythons

Es un juego de dos jugadores con información completa, y se resuelve con **programación dinámica sobre intervalos**. La clave es calcular la **diferencia de puntos** entre quien mueve y su rival, no la puntuación de cada uno.

Sea `v[i]` el valor del cristal `i`. Definimos `dp[i][j]` como la **máxima diferencia** que consigue quien mueve si solo quedan los cristales de `i` a `j`. Puede coger un extremo:

- **El izquierdo:** gana `v[i]` y el rival juega en `[i+1, j]`, con diferencia `dp[i+1][j]` a su favor. Resultado: `v[i] - dp[i+1][j]`.
- **El derecho:** resultado `v[j] - dp[i][j-1]`.

Por tanto, `dp[i][j] = max(v[i] - dp[i+1][j], v[j] - dp[i][j-1])`. El caso base es el intervalo vacío (`i > j`), que vale `0`; el de un cristal sale solo de la fórmula (`v[i] - 0`). La respuesta es `dp[0][N-1]` y puede ser negativa, porque hay que coger un cristal aunque sea negativo.

Un greedy (coger el mayor extremo) no sirve, porque un cristal grande puede dejar al rival uno todavía mejor.

Proponemos **memoización (top-down)**: una función recursiva `solve(i, j)` que calcula `dp[i][j]` y guarda los resultados en una tabla, de modo que cada intervalo se resuelve una sola vez:

- Si `i > j`, devuelve `0`.
- Si el estado ya está calculado, devuelve el valor guardado.
- Si no, calcula `max(v[i] - solve(i+1, j), v[j] - solve(i, j-1))`, lo guarda y lo devuelve.

Hay `O(N^2)` estados con transición `O(1)`, así que el coste es `O(N^2)` en tiempo y memoria. La recursión llega a profundidad `N`, así que hay que subir el límite con `sys.setrecursionlimit`.

También se puede resolver sin recursión con un enfoque **bottom-up**, calculando los intervalos por longitud creciente. Como cada estado solo depende de intervalos más cortos, la memoria se reduce a `O(N)`.